

Git Hook 기반 클라우드 DevSecOps 보안 게이트웨이



3조 깃반인 - 이준서(C) 강○○ 김○○

Mentor - 박○○(퍼○○○) | 문○○(슬○○○)

CONTENTS

- 01 팀원 소개 및 일정
- 02 주제 선정 배경
- 03 프로젝트 소개
- 04 프로젝트 구성
- 05 시연영상 및 개선 방향
- 06 출처 및 질의응답

1. 팀원 소개 및 일정

01

팀 소개



= GitHub + 일반인



이준서 (C)

- Git Hook 자동화 프레임워크 설계
- 사용자 로컬 환경 연동 및 설치 스크립트 최적화
- 최종 시스템 통합 및 프로젝트 발표



강○○

- git push / git pull 검증 시스템 기능 구현
- 보안 도구별 스캔 로직 세부 파라미터 최적화
- 기술 문서화 및 PPT 작성



김○○

- 검증 시스템 오류 검증 및 수정
- 터미널 UI 기반 경고 및 알림 기능 구현
- PPT 시각화 자료 작성

MON	TUE	WED	THU	FRI	SAT	SUN
6/21	22	23	24	25	26	27
	프로젝트 시작!					
28	29	30	7/1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18

2026.06.22 ~ 06.24

1주

- 팀원 구성 및 주제 선정
- 기획서 작성
- 기술 스택 조사

MON	TUE	WED	THU	FRI	SAT	SUN
6/21	22 프로젝트 시작!	23	24	25	26	27
28	29	30	7/1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18

2026.06.25 ~ 07.03

2주

- 자동화 구조 기획
- 핵심 기능 설계 및 구현

MON	TUE	WED	THU	FRI	SAT	SUN
6/21	22 프로젝트 시작!	23	24	25	26	27
28	29	30	7/1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18

2026.07.04 ~ 07.07

3주

- 구현기능 통합 테스트
- 트러블 슈팅
- 마무리 작업

MON	TUE	WED	THU	FRI	SAT	SUN
6/21	22 프로젝트 시작!	23	24	25	26	27
28	29	30	7/1	2	3	4
5	6	7	8	9	10	11
12	13	14 발표	15	16	17	18

2026.07.08 ~ 07.14

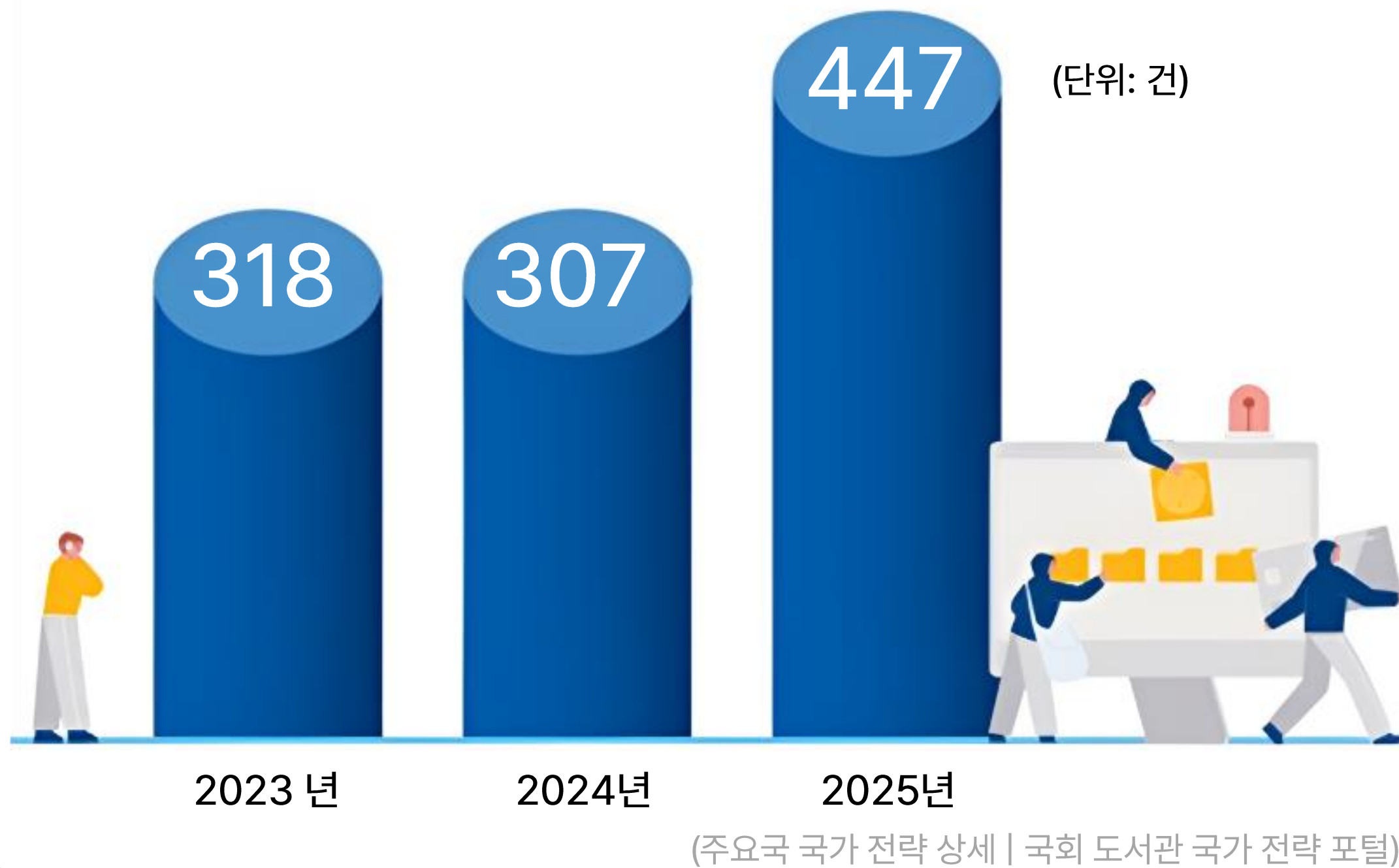
4주

- 시연 영상 준비
- PPT 제작
- 최종 발표 대본 작성

2. 주제 선정 배경

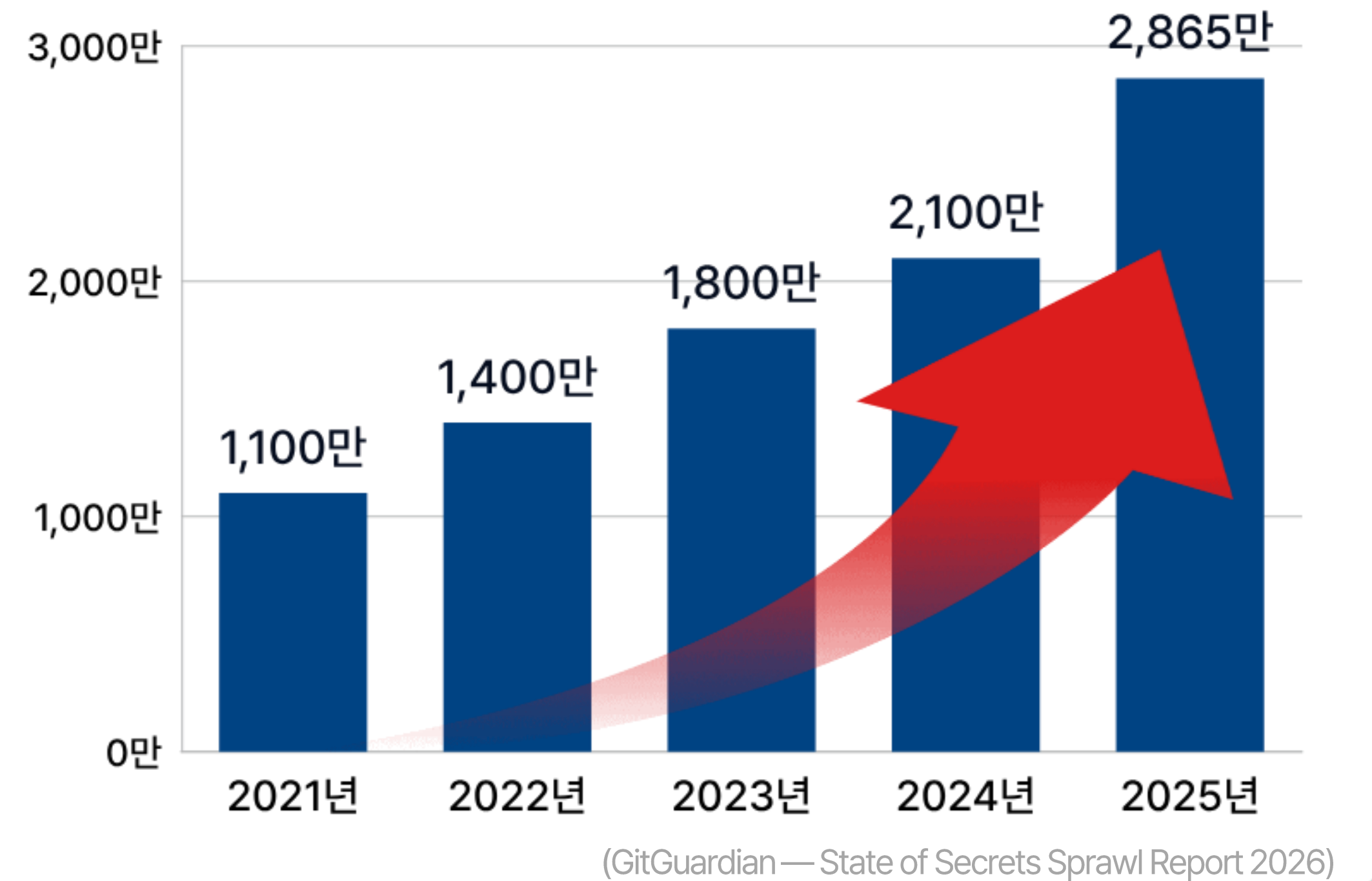
주제 선정 배경

최근 3년간 개인정보 유출 신고 현황



2025년 개인정보 유출 신고 건수가 최대치를 기록

GitHub Secret 유출, 매년 폭발적 증가!!!



2025년 Secret 유출건수가 역대 최대치를 기록

최근 3년간 개인정보 유출 신고 현황

2025년도 역대 최대치 기록!!!

2023년

2024년

2025년



2025년 개인정보 유출 신고 건수가 최대치를 기록

GitHub Secret 유출, 매년 폭발적 증가!!!

3,000만

2,865만

2,100만

0만

2021년

2022년

2023년

2024년



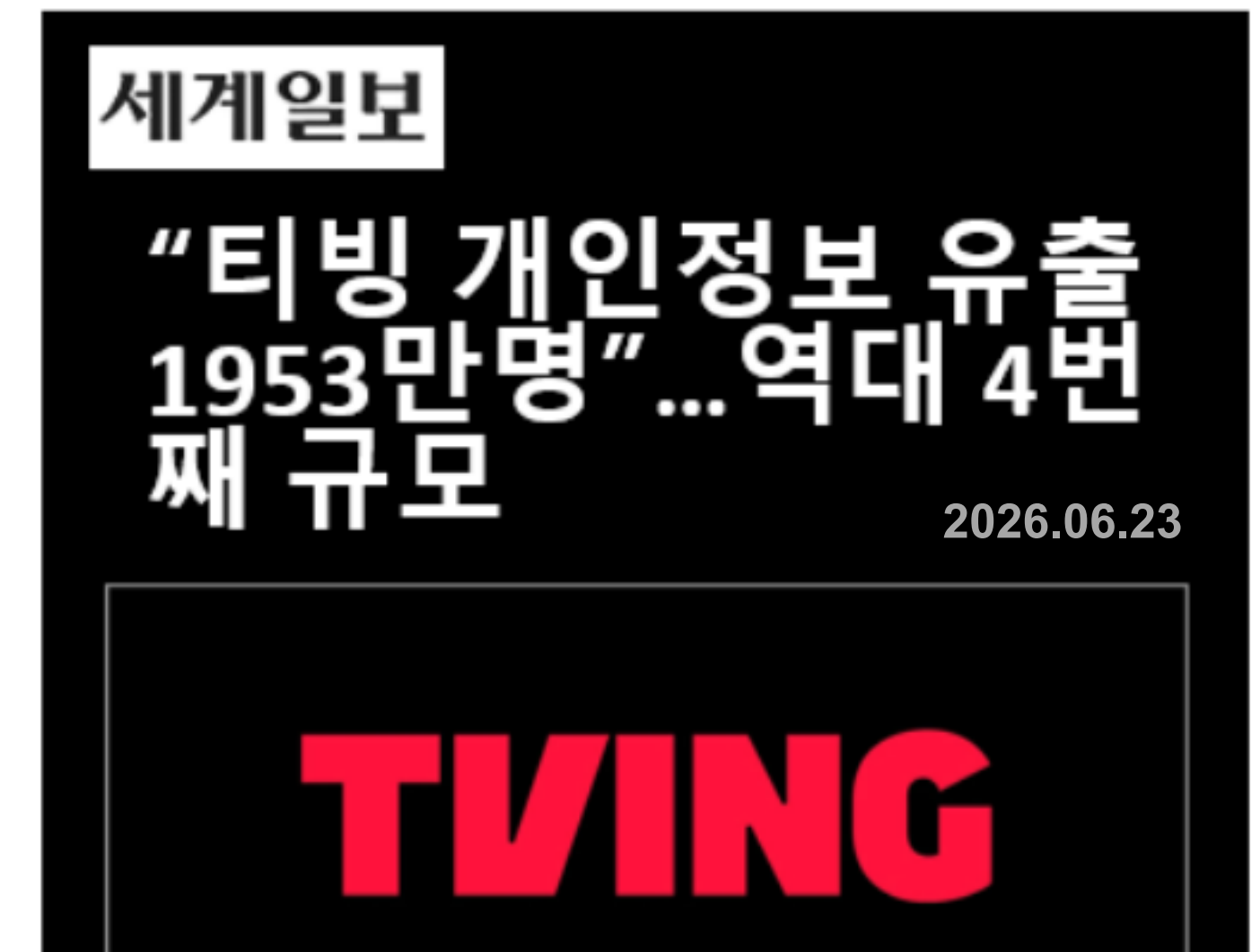
2025년 Secret 유출건수가 역대 최대치를 기록



Github RCE 취약점



Github OAuth 토큰 탈취



Github 자격증명 탈취 및 AWS 액세스 키 유출



STEP. 1 탐지



취약점 탐지

발생할 수 있는 사고원인
즉각 탐지

STEP. 2 실행



로컬 기반 실행

외부 프로그램 없이
단독 실행

STEP. 3 차단



사전 차단

데이터가 외부로 이동하기 전
즉시 차단



사후 조치(키 폐기·교체)는 이미 **사고가 일어난 뒤의 대처**입니다.
코드가 컴퓨터를 떠나기 전에 작동하여 원격지(GitHub)
유출 위험으로부터 안심할 수 있는 환경을 만듭니다.

취약점 탐지

발생할 수 있는 사고원인
즉각 탐지

로컬 기반 실행

외부 프로그램 없이
단독 실행

사전 차단

데이터가 외부로 이동하기 전
즉시 차단

개발 / DevOps 조직



01

애플리케이션 개발 및
배포를 담당하는 엔지니어

클라우드 인프라 실무 개발자



02

AWS, Azure 등 클라우드
자원을 구성하는 현장 개발자

소규모 팀

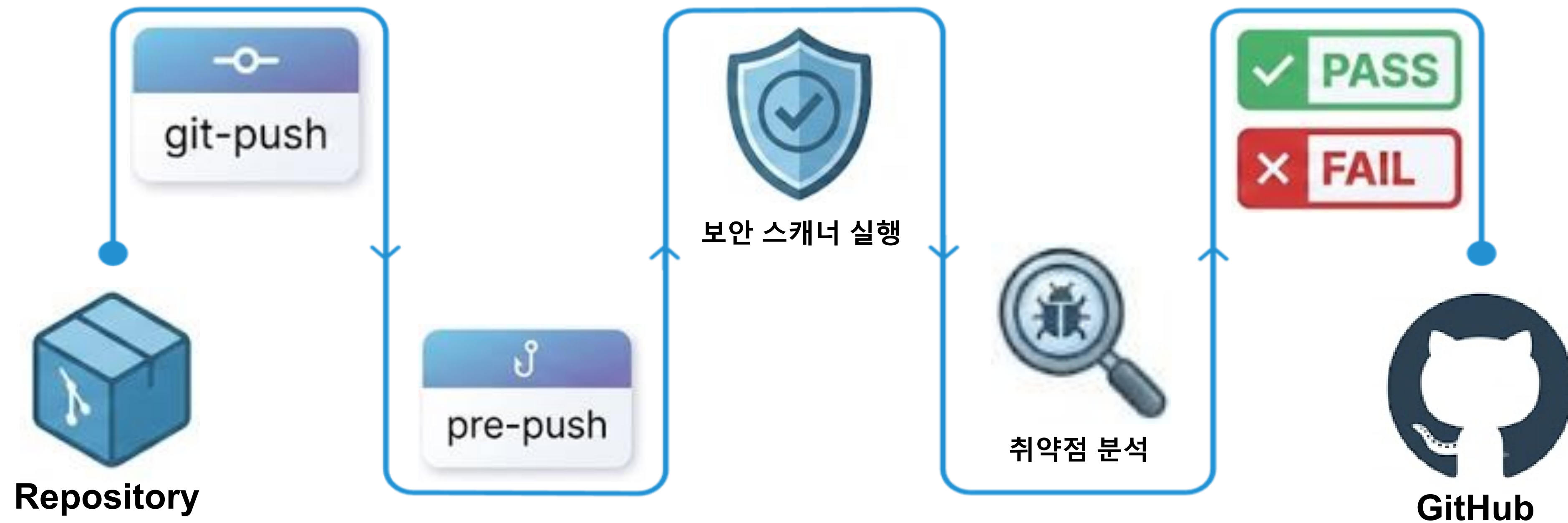


03

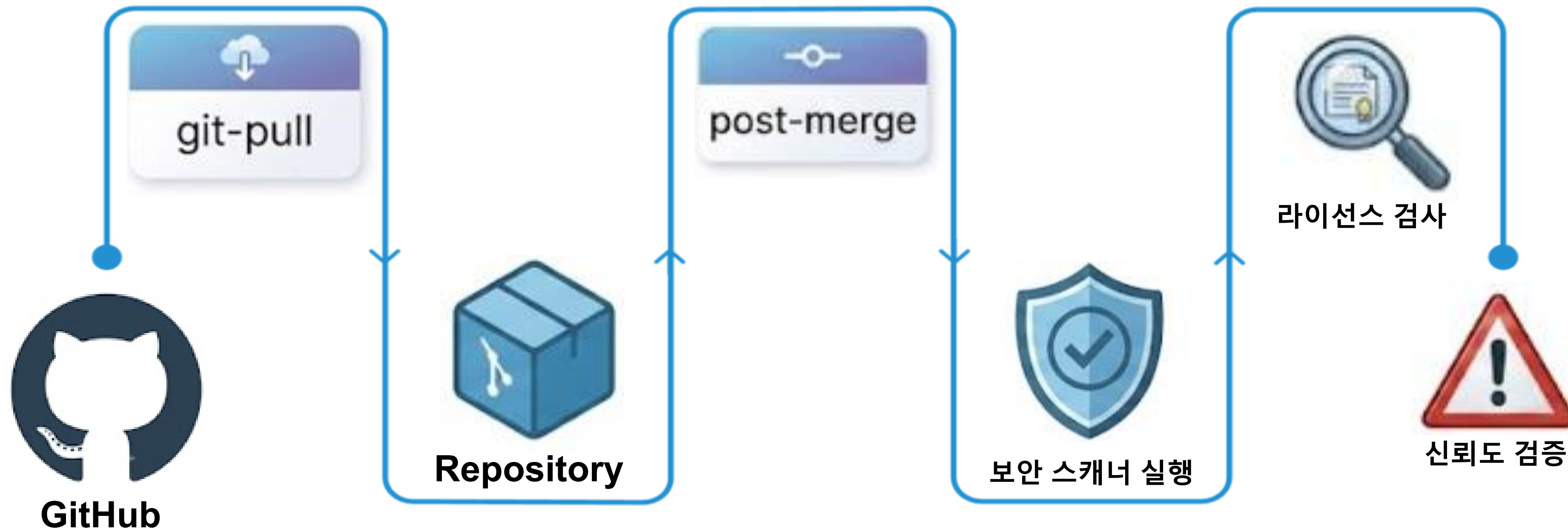
소규모 다기능
스타트업 팀

3. 프로젝트 소개

pre-push 흐름도



post-merge 흐름도





Shift-Left
Security



원클릭 자동화 시스템

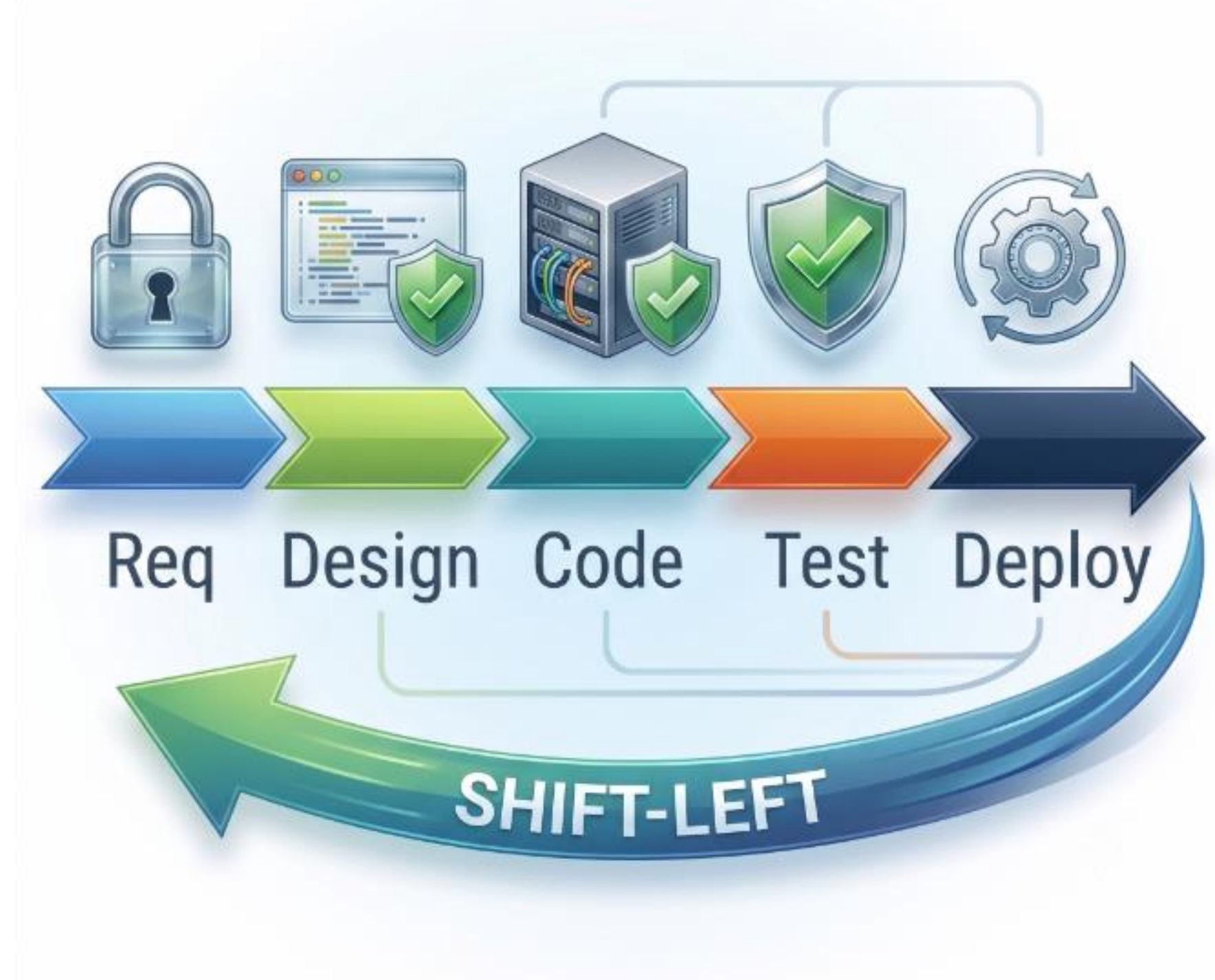


로컬 기반
경량화 아키텍처



통합 오케스트레이터

Shift-Left Security



보안 검증을 '**사후**'가 아닌 '**사전**'으로 앞당기는 것

원클릭 자동화 시스템



개발자 작업



Git Hook 자동실행



Git 저장소

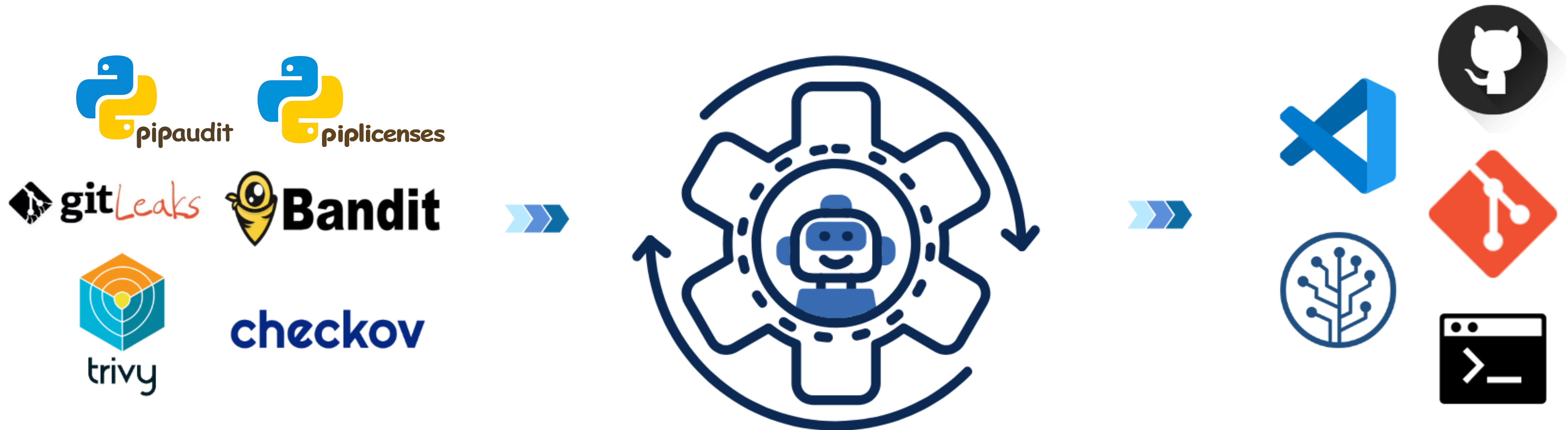
전체 보안 검증 체계가 **자동**으로 갖춰지는 **원클릭** 자동화 시스템

로컬 기반 경량화 아키텍처



외부 API 통신 없이, 로컬 자원만으로 고속 보안 검사

보안 도구 통합 관리



복잡한 도구들을 **Orchestrator** 하나로 **통합 제어**

4. 프로젝트 구성



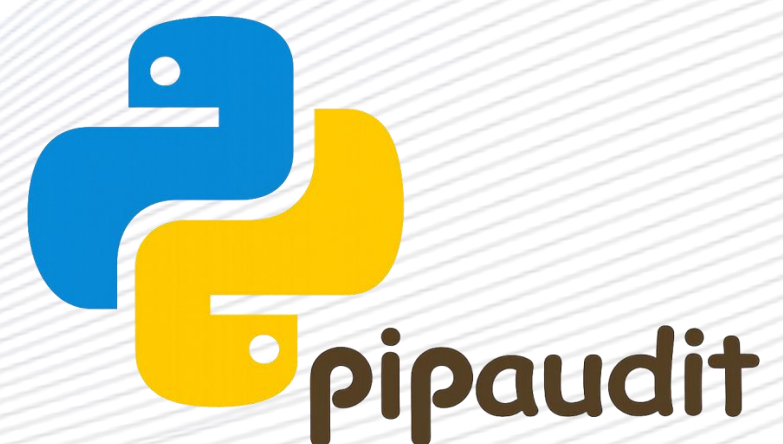
Pyhon

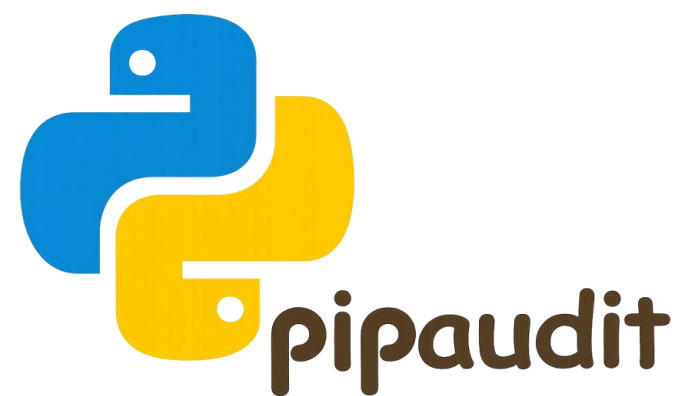


Git Hub



checkov

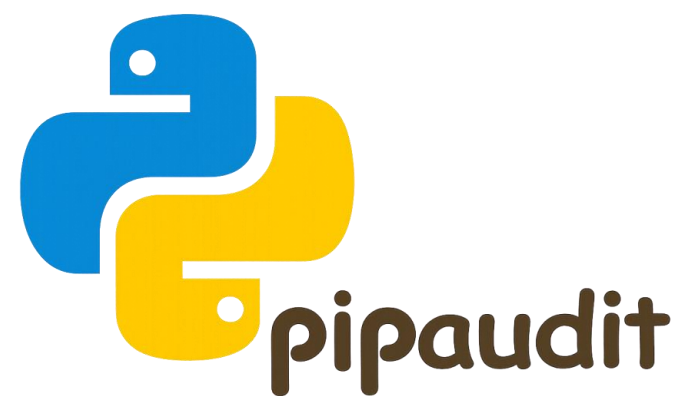




소스코드 내 민감 정보 유출 방지 도구

Git 커밋 기록을 스캔하여 API 키, 비밀번호, 인증 토큰 탐지

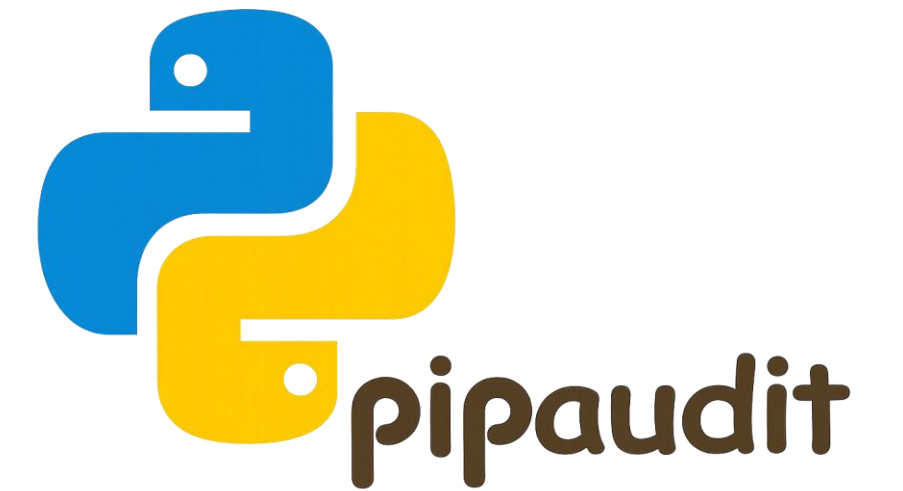
실수로 하드코딩된 중요 자격 증명이 GitHub 등에 공개되는 것을 차단



파이썬 소스코드 정적 보안 분석(SAST) 도구

파이썬 코드 자체를 분석하여 흔히 발생하는 코딩 실수 및 보안 취약점 발견

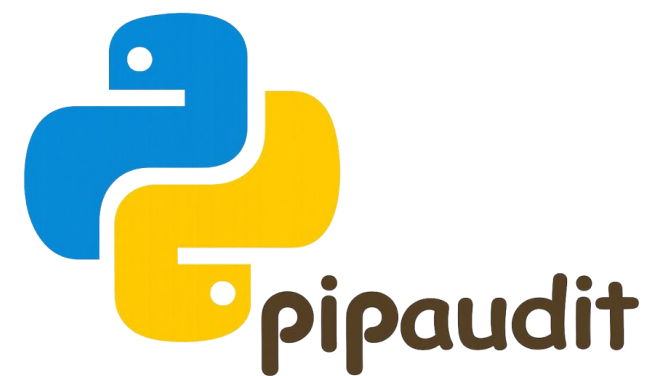
셸 인젝션 위험, 취약한 암호화 알고리즘 사용 등 코드 레벨의 약점 탐지



파이썬 패키지(의존성) 취약점 검사 도구

프로젝트에 사용된 파이썬 라이브러리의 보안 약점 검사

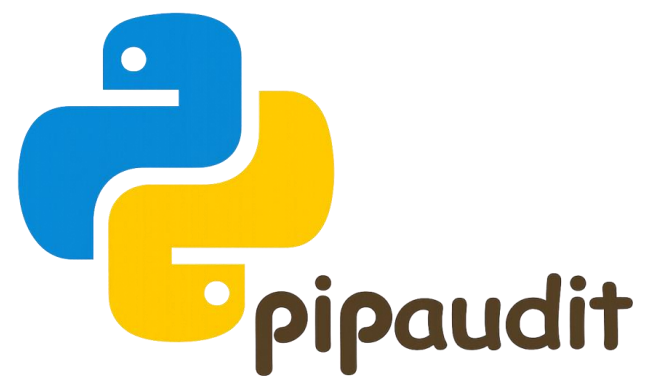
PyPI(파이썬 패키지 인덱스)의 알려진 취약점 데이터베이스와 대조하여
오픈소스 공급망 공격을 예방



파이썬 패키지 라이선스 관리 및 법적 리스크 방지 도구

사용 중인 파이썬 라이브러리들의 라이선스 종류(MIT, GPL 등) 전수 조사

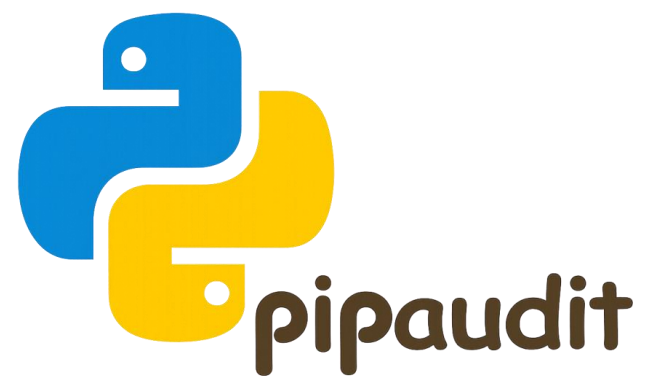
기업 보안 정책이나 오픈소스 법적 분쟁 소지가 있는 라이선스 식별



컨테이너 및 인프라(IaC) 종합 보안 취약점 진단 도구

도커 이미지, OS 패키지, 오픈소스 라이브러리의 보안 취약점 및 라이선스 위험성 분석

Terraform, Dockerfile, docker-compose 등 코드형 인프라(IaC)의 보안 설정 오류 검사



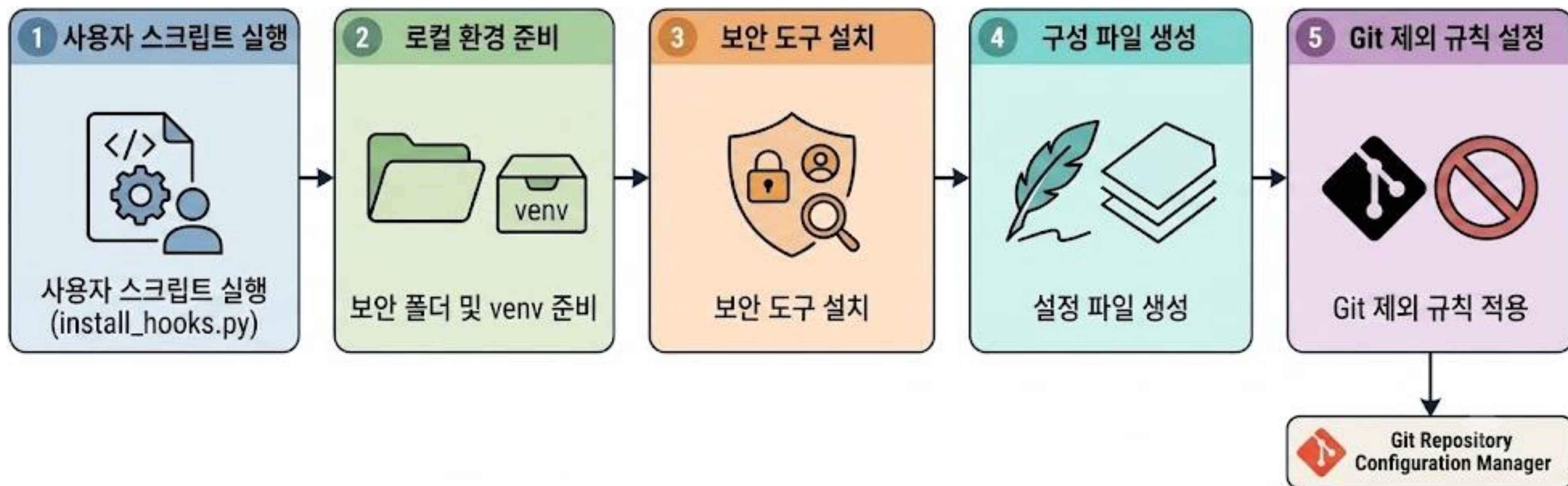
checkov

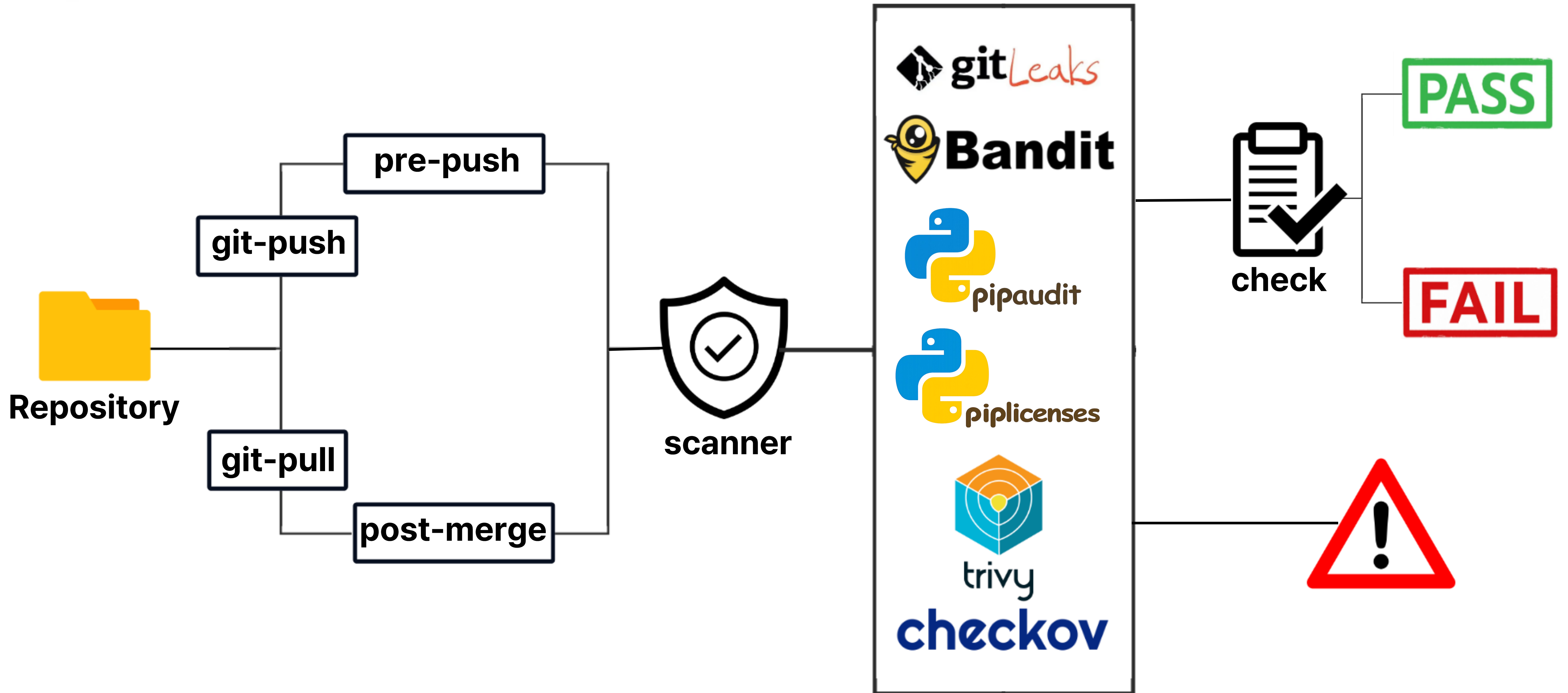
인프라 코드(IaC) 보안 설정 검증 도구

테라폼(Terraform), 쿠버네티스, 클라우드 설정 파일의 보안 취약점 분석

배포 전 단계에서 인프라의 정책 위반 및 보안 가이드라인 준수 여부 점검

보안 스캐너 설치 자동화 단계






```
# 가상환경 무결성 검증 및 자동 복구
```

```
venv_missing = not target_executable.exists()
```

```
venv_broken = (not venv_missing) and (not _python_is_usable(str(target_executable)))
```

```
if venv_broken:
```

```
    print("기존 보안 가상환경이 손상되었습니다. 정리하고 새로 만듭니다.")
```

```
    shutil.rmtree(venv_dir, ignore_errors=True)
```

```
if venv_missing or venv_broken:
```

```
    base_python_cmd = get_target_python()
```

```
    subprocess.run( base_python_cmd + ["-m", "venv", str(venv_dir)], check=True )
```

프로젝트 독립형 보안 가상환경

.venv-security 폴더에 독립된

가상환경 구축

무결성 검사 후 가상 환경 손상 시

자동 복구

```
def install_hook(hooks_dir: Path, hook_name: str, content: str) -> None:
    hook_path = hooks_dir / hook_name
    backup_path = hooks_dir / f"{hook_name}.bak"

    # [핵심] 기존 훅이 있고 백업본이 없을 때만 백업 진행 (무한 덮어쓰기 방지)
    if hook_path.exists() and not backup_path.exists():
        shutil.copy2(hook_path, backup_path)
        print(f"기존 {hook_name} 훅을 .bak 으로 백업했습니다.")

    hook_path.write_text(content, encoding="utf-8")
    make_executable(hook_path)
```

Git Hook 자동화 및 안전한 덮어쓰기 방지

pre-push, post-merge 훅을 자동 설치
기존 훅이 존재할 경우, .bak 파일로 백업


```
# 다운로드한 파일의 무결성 검증 (SHA256)
file_bytes = response.read()

if expected_hash:
    actual_hash = hashlib.sha256(file_bytes).hexdigest()
    if actual_hash.lower() != expected_hash.lower():
        print("체크섬 불일치! 다운로드가 손상되었거나 변조되어 설치를 중단합니다.")
        return # 설치 중단

print("체크섬 검증 완료 (SHA256 일치).")
```

무결성 검증이 포함된 바이너리 자동 다운로드

보안 도구 다운로드 시 체크섬을 검증하여,
파일 변조/손상 여부 확인 후 보안도구
설치 확정

```
def main() -> int:
    mode_arg = sys.argv[1] if len(sys.argv) > 1 else ScanMode.PRE_PUSH
    repo_root = get_repo_root()

    safe_tmp = Path(repo_root) / "security-tools" / ".tmp"
    if safe_tmp.exists():
        shutil.rmtree(safe_tmp, ignore_errors=True)
    safe_tmp.mkdir(parents=True, exist_ok=True)

    SCAN_ENV["TMP"] = str(safe_tmp)
    SCAN_ENV["TEMP"] = str(safe_tmp)
    SCAN_ENV["TMPDIR"] = str(safe_tmp)

    report = run_post_merge_scan(repo_root) if mode_arg == ScanMode.POST_MERGE else
        run_pre_push_scan(repo_root)
    _write_vulnerability_log(repo_root, report)
    print_report(report)
    return 0 if report.overall_passed else 1

if __name__ == "__main__":
    sys.exit(main())
```

보안 스캐너 구현 로직

Git과 연동된 로컬 저장소의
최상위로 찾아가 검사, 보안 도구의
저장 위치를 구속 받지 않도록 구현

6가지 기능의 통합 오케스트레이터 구현

```
def run_pre_push_scan(repo_root: str) -> ScanReport:
    report = ScanReport(hook_type=ScanMode.PRE_PUSH)
    _prepare_shared_state(repo_root)
    scan_steps = [
        ("Gitleaks - 시크릿/인증정보 탐지", run_gitleaks),
        ("Bandit - Python 코드 취약점 분석", run_bandit),
        ("pip-audit - CVE 의존성 취약점 검사", run_pip_audit),
        ("pip-lic - 라이선스 정책 검사", run_license_scan),
        ("Trivy - IaC 설정 오류 검사", run_trivy),
        ("Checkov - IaC 정책 위반 검사", run_checkov)
    ]
    report.results = _execute_steps(repo_root, scan_steps)
    return report
```

```
def run_post_merge_scan(repo_root: str) -> ScanReport:
    report = ScanReport(hook_type=ScanMode.POST_MERGE)
    _prepare_shared_state(repo_root)

    report.merge_risk = evaluate_merge_risk(repo_root)

    # 💡 새로 가져온 오픈소스 의존성 신뢰도 분석 진행
    print(colored("\r 🔄 가져온 오픈소스 의존성 신뢰도 분석 중...", Color.DIM), end="", flush=True)
    report.os_trust_warnings, report.os_trust_safe = check_new_opensource_trust(repo_root)

    print(colored("\r ✓ 가져온 오픈소스 의존성 신뢰도 분석 완료! ", Color.GREEN) + " "*20)
    print()

    scan_steps = [
        ("Gitleaks - 시크릿/인증정보 탐지", run_gitleaks),
        ("Bandit - Python 코드 취약점 분석", run_bandit),
        ("pip-audit - CVE 의존성 취약점 검사", run_pip_audit),
        ("pip-lic - 라이선스 정책 검사", run_license_scan),
        ("Trivy - IaC 설정 오류 검사", run_trivy),
        ("Checkov - IaC 정책 위반 검사", run_checkov)
    ]
    report.results = _execute_steps(repo_root, scan_steps)
    return report
```

```
def _execute_steps(repo_root: str, scan_steps: list) -> list:
    results = [None] * len(scan_steps)

    # [핵심] 여러 보안 도구를 스레드 풀을 이용해 병렬로 동시 실행
    with concurrent.futures.ThreadPoolExecutor(max_workers=len(scan_steps)) as executor:
        future_to_idx = {
            executor.submit(fn, repo_root): i for i, (_, fn) in enumerate(scan_steps)
        }
        for future in concurrent.futures.as_completed(future_to_idx):
            idx = future_to_idx[future]
            results[idx] = future.result()

    return results
```

멀티스레딩 기반 병렬 보안 검사

ThreadPoolExecutor를 활용해 보안 도구를 병렬 실행, 검사 요소 시간을 획기적으로 단축


```
def _score_pypi_package(name: str, spec: str, pinned: bool) -> tuple[int, list[str]]:  
    score = 100  
    popular = ["requests", "flask", "django", "numpy", "pandas", "boto3"]  
  
    # [핵심] 레벤슈타인 거리 알고리즘으로 타이포스쿼팅(Typo-squatting) 탐지  
    for p in popular:  
        if norm != p and _levenshtein(norm, p) <= 2:  
            score -= 30  
            reasons.append(f"유명 패키지명({p})과 유사함")  
            break  
  
    # [핵심] 버전 고정 여부 확인  
    if not pinned:  
        score -= 20  
        reasons.append("버전 미고정")
```

오픈소스 신뢰도 및 공급망 공격 방어

레벤슈타인 거리 알고리즘으로 타이포스
쿼팅 탐지
패키지의 라이선스 존재 여부, 업데이트
방치 여부, 다운로드 수 등을 종합
평가하여 경고

```
try:
    proc = subprocess.run([project_python, "-m", "pip", "freeze"], capture_output=True,
                           text=True, timeout=15)
    if proc.returncode != 0 or not proc.stdout.strip():
        return None, False

    header = f"# 자동 생성됨 - '{project_python}' 환경을 기준으로 매번 새로 생성됨\n"
    generated_path.write_text(header + proc.stdout, encoding="utf-8")
    return str(generated_path), True
except Exception:
    pass
```

작업환경 라이선스 탐지 자동화

의존성 파일이 없으면 현재 작업환경에서
pip freeze로 임시 생성 후 취약점과
라이선스 검사


```
@dataclass
class ScanIssue:
    tool: str; file_path: str; line: str; severity: str; message: str

@dataclass
class ScanResult:
    tool: str; passed: bool; message: str
    issues: list[ScanIssue] = field(default_factory=list)
    skipped: bool = False; skip_reason: str = ""

@dataclass
class ScanReport:
    hook_type: str
    results: list[ScanResult] = field(default_factory=list)

    @property
    def overall_passed(self) -> bool:
        return all(r.passed or r.skipped for r in self.results)
```

보안 도구의 출력 형식의 통합

서로 다른 출력 형식을 하나로 통일하여
파일의 몇 번째 줄에서 어떤 문제를
발견했는지 출력

5. 시연 영상 및 개선 방향



안녕하세요.



개발자 부담 감소

개발자 개인의 수동 코드 검수 책임·부담 감소



검수 비용 절감

비용 대비 효과가 큰 보안 투자, 검수 시간·비용 절감



빠른 온보딩

추가적인 교육 없이 누구나 쉽게 사용 가능



앱 및 대시보드 구현



다양한 언어 분석 도구



룰 커스텀 튜닝 기능



보안 습관 형성 - 평범한 실수 방지 및 개인 보안 습관 내재화



진정한 협업 - 지속적인 소통을 통한 의견 조율과 목표 달성



명확한 방향성 - 초기에 목표를 세워둌으로써 구체적인 기준 생성

6. 출처 및 질의응답

- **GitGuardian — State of Secrets Sprawl Report 2026 (p.10)**
<https://www.gitguardian.com/state-of-secrets-sprawl-report-2026>
- **주요국 국가 전략 상세 | 국회 도서관 국가 전략 포털 2026 (p.10)**
<https://nsp.nanet.go.kr/plan/main/detail.do?nationalPlanControlNo=PLAN0000063175>
- **데일리시큐 2026 (p.12)**
<https://www.dailysecu.com/news/articleView.html?idxno=206540>
- **세계일보 2026 (p.12)**
<https://www.segye.com/newsView/20260623500281>
- **EST SECURITY 2022 (p.12)**
<https://blog.alzac.co.kr/4648>
- **오픈소스 도구 공식 문서**
Gitleaks · Bandit · pip-audit · pip-licenses · Trivy · Checkov 각 공식 문서

Q&A

감사합니다

LET'S BUILD A SECURE FUTURE TOGETHER!

